DSA STUDY BLUEPRINT SERIES

112. Topological Sorting in Graph using DFS

Ordering Dependency graphs using DFS Stacks

Section 10 • C++ Core Data Structures & Algorithms

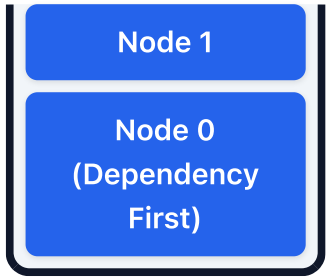
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Topological Sort:** Ordering DAG vertices such that for edge $u \rightarrow v$, u comes before v.
- Push nodes to stack after visiting all neighbors.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
void topoDFS(int u, vector>& adj, vector& vis, stack& st) {
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) topoDFS(v, adj, vis, st);
    st.push(u); // Push to stack
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	O(V + E)
Space Complexity	O(V)

Key Practices & Gotchas

- **Important:** Only applicable to Directed Acyclic Graphs (DAGs).
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.